



Project Report on

AirSense: Intelligent Air Quality Prediction

Submitted in Big Data Analytics and Business Intelligence Course, CSET 371

**Bachelor of Technology
Of
Bennett University, Greater Noida**

Jinia	E23CSEU1015
Navya Agarwal	E23CSEU0954
Aayush	E23CSEU1018



**BENNETT UNIVERSITY (THE TIMES GROUP), GREATER
NOIDA**

Under the guidance of

Faculty Name
Dr. Aditya Bhardwaj

School of Computer Science Engineering and Technology (SCSET)



DECLARATION

We hereby declare that the project report entitled “**AirSense: Intelligent Air Quality Prediction**”, **has not been copied or plagiarized from any other source**, and all information, data, and materials used in this report have been **properly acknowledged and cited in the text**. The work has not been submitted previously for any other course project.

PLAGIARISM REPORT CERTIFICATE

Further, the plagiarism similarity index of the report from Turnitin is _____ %, AI plag is _____ % which is within the permissible limit of Turnitin and AI must be 0%. All sources of information used in the report have been properly acknowledged and cited.

Teamwork and Collaboration

Team member name	Enrollment No	Individual Contribution in Project
Jinia	E23CSEU1015	PySpark (Model training) & Databricks
Aayush	E23CSEU1018	PySpark (Model training) & Databricks
Navya Aggarwal	E23CSEU0954	PowerBI



Chapter 1: Problem Analysis Introduction.....	1 to 5
1.1 Background of the Study	1
1.2 Problem Statement	2
1.3 Objectives of the Project	3
1.4 Scope of the Project	4
1.5 Review of Existing Systems.....	5
Chapter 2: Project Design and Methodology.....	9 to 15
2.1 System Overview	9
2.2 System Architecture Diagram Self-Drawn.....	10
2.3 Proposed Methodology Flow Chart Self-Drawn.....	11
2.4 Algorithms Used	12
2.5 Tools and Technologies Used	13
2.6. Experimental Setup Environment.....	14
2.6. Dataset Description.....	15
Chapter 3: Project Implementation	16 to 21
3.1 Development Environment	16
3.2 Module Description	17
3.3 Code Implementation	18-19
3.4 System Interface / Screenshots	20-21
Chapter 4: Project Results and Discussion.....	22 to 28
4.1 Experimental Setup	22
4.2 Output Results	23-25
4.3 Performance Analysis	26-27
4.4 Discussion	28
Chapter 5: Project Conclusion and Presentation.....	29 to 30
5.1 Conclusion	29
5.2 Limitations	29
5.3 Future Scope	30
5.4 References.	30
Appendix A – Additional Screenshots	31 onwards

Chapter 1

Problem Analysis Introduction

1.1 Background of the Study

Improvement in air quality has been one of the key environmental and public health issues faced by the world in the twenty-first century. With fast-paced development, increase in population, rapid urbanization, and high usage of fossil fuels, there has been a considerable decline in the quality of air in various parts of the country as well as other countries. Particulate Matter (PM_{2.5} and PM₁₀), Nitrogen Dioxide (NO₂), Sulphur Dioxide (SO₂), Carbon Monoxide (CO), Ozone (O₃), Benzene, Toluene, and Xylene are some of the pollutants that are harmful to human health and lead to respiratory disorders, heart diseases, and early deaths.

Air Quality Index (AQI) is a standardized numerical scale used to inform people about the levels of air pollution. The CPCB of India classifies the categories of AQI into good, satisfactory, moderate, poor, very poor, and severe. The AQI prediction plays an important role as it helps decision-makers as well as the common public to take precautionary steps accordingly.

The conventional way of monitoring AQI involves the extensive use of physical sensors and manual data collecting processes, making it costly, geographically restricted, and vulnerable to missing and delayed data points. However, the availability of extensive historical air quality data has enabled more efficient AQI prediction through machine learning techniques using large sets of data.

Apache Spark, along with its library PySpark MLlib, is an efficient distributed computing engine for handling large environmental datasets. This paper proposes the use of machine learning algorithms in PySpark MLlib to develop AQI category classifiers using multi-dimensional pollutant data collected in several cities and monitoring stations in India. Two datasets are used in this study: one consists of city and station-level data collected in five major cities in India—Delhi, Mumbai, Chennai, Kolkata, and Bangalore; the other one includes data collected at state and area levels across India.

1.2 Problem Statement

The problem of air pollution is one of the leading environmental issues in India, especially in the urban regions where toxic substances have been detected in extremely alarming quantities. The presence of an AQI measurement system does not guarantee an efficient way to predict air quality due to various constraints.



The key problems addressed in this project are as follows:

- The current AQI monitoring system uses sensors, thus making it location-specific and failing to provide information for vast regions where no sensors exist.
- The conventional method of collecting data manually and analyzing the data using statistical tools is unable to handle the large number of records that come from multiple sources within the environment, especially those recorded at several thousand monitoring stations.
- The absence of a comparative study to determine the performance of several machine learning models using heterogeneous air quality data sets of various spatial granularities is another challenge.
- No previous study has used distributed computing systems like Apache Spark to process massive amounts of data over the years, across cities and pollutants.
- No study has been conducted to find out the difference in the effectiveness of machine learning models with regard to the variation in features available in the data set.

Such issues are addressed by this study through the application and comparison of five classification techniques, namely logistic regression, decision tree, random forest, one-vs-rest(logistic regression), and naive bayes classifiers on two structurally distinct datasets related to air quality data through the use of Apache PySpark platform.

1.3 Objectives of the Project

The main objective of this project is to develop a scalable machine learning pipeline using PySpark for predicting and classifying AQI values in various cities in India. The individual objectives include:

- For creating a setup for Apache Spark (PySpark) that can efficiently process massive air quality datasets in a distributed manner.
- For preprocessing and feature engineering on two distinct types of air quality data sets, namely the City/Station level and the State/Pan India level datasets.
- For implementing five different algorithms in classification, specifically Logistic Regression, Decision Tree, Random Forest, OneVsRest (Logistic Regression) and Naïve Bayes algorithms using PySpark MLlib.
- For testing all the above implementations using common metrics such as Accuracy, Weighted Precision, Weighted Recall, F1 Score and Loss.
- For carrying out a comparative analysis of all models' performances on two datasets to find the best algorithm for AQI classification.
- For interpreting confusion matrices and analyzing results to gain insight into misclassification errors and quality of datasets effect on predictions.
- For obtaining valuable recommendations to improve the overall AQI classification system.



1.4 Scope of the Project

Introduction

This project aims to predict the classification of AQI categories using machine learning models created using the Apache PySpark framework. The scope of this project is:

Geographical Scope

The geographical scope includes air pollution data from five major metropolitan cities of India - Delhi, Mumbai, Chennai, Kolkata, and Bangalore for the pollutant-level data set (Dataset 1) and a larger geographical scope including various states and cities across India for the State-Level AQI Dataset (Dataset 2).

Data Scope

The first data set includes day and hour level readings of pollutants (12 different pollutants - PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene) apart from the AQI value and its categorization into AQI Buckets. The second data set includes Date, State, Area name, Count of Monitoring Stations, Prominent Pollutant, AQI Value, and Air Pollution Status. Regression analysis to predict AQI values is outside the scope of this project.

Technical Scope

The technical scope involves using Python programming language and Apache Spark version 3.4.1 in Google Colab. It uses MLlib functions for feature engineering (StringIndexer, VectorAssembler), model training, and evaluation. Five classification models are part of the technical scope and Deep Learning models, Time-Series Analysis models, and Stream Processing are out of scope.

Evaluation Scope

Evaluation of the model is performed using standard metrics of multiclass classification such as accuracy, precision, recall, F1 score, and classification loss. Evaluation using bar graphs for each dataset is also considered in this analysis. Optimization of hyperparameters and cross-validation are not considered in this study.

1.5 Review of Existing Systems

A lot of studies have been carried out concerning research related to air quality monitoring, AQI forecasting, and using machine learning models for analyzing environmental data. The present study investigates current practices and the available literature related to this topic in order to identify success stories and limitations that motivate us to carry out this research.



1.5.1 Traditional AQI Monitoring Systems

The CPCB in India runs CAAQMS systems in certain urban centers where air quality needs to be monitored regularly. The monitoring takes place in real-time through the sensors installed at CAAQMS stations, though there exist shortcomings regarding its coverage areas and cost of implementation. Manual computation is required for determining AQI sub-indexes of each individual pollutant and the AQI is not used for future forecasting.

1.5.2 Machine Learning Techniques

Several studies have used machine learning approaches in AQI prediction. From the study by Kumar & Goyal (2011), the use of statistical regression models was reported for predicting PM10 concentrations in Delhi, and traffic factors and meteorology were revealed as significant predictors. In their study, Gu et al. (2020) noted that the RF model predicted PM2.5 levels better than the linear regression and SVM models due to its ability to capture non-linear relationships between pollutants. Decision Trees have been commonly used in classification AQI values due to interpretability; however, they can suffer from overfitting the environmental data noise.

The Naive Bayes approach has been used in environmental problems to some extent. The assumption of Naive Bayes models of feature independence rarely holds in air pollution cases since the pollutants are correlated. Logistic Regression models are quick and cheap to implement but assume linear boundaries of decision-making, thereby failing in multi-pollutant cases.

1.5.3 Deep Learning and Ensemble Methods

Further advancements involve deep learning algorithms like LSTM neural networks to predict time series AQI levels. Zhao et al. (2019) found that LSTM-based neural networks were capable of capturing the temporal dependency of pollutants for predicting further ahead into the future. Yet, deep learning algorithms are computationally expensive and need a substantial amount of sequential data. The ensemble algorithm has proven efficient at modeling tabular environmental data.

1.5.4 Role of Big Data and Distributed Computing in AQI Analysis

With the advent of an enormous number of sensor data regarding environmental aspects, big data technology has gained more relevance in the field. In several environmental data monitoring studies, Apache Spark was employed because of its suitability for handling distributed computing tasks. For



instance, Chen et al. (2018) presented a Spark-enabled framework for the analysis of air quality data, which led to a considerable reduction in processing times for large-scale datasets. Yet, research comparing various ML classification techniques on PySpark pipelines is limited.

1.5.5 Gaps in Existing Systems

After analyzing existing systems, it can be seen that the following are the major gaps in the existing systems that this current project intends to fill:

- The majority of the existing research work has been done for one particular city or area only, without considering the performance of models on different datasets having different spatial scales.
- There have not been any studies that have performed comparative analysis of various ML classifiers on the same AQI datasets through a common PySpark pipeline.
- There has not been much research work carried out on how the feature richness of a dataset affects the performance of the classifier used.
- There are not enough scalable and reproducible PySpark pipelines for India's AQI datasets.

Chapter 2

Project Design and Methodology

2.1 System Overview

The developed model is aimed at predicting and classifying Air Quality Index (AQI) classes by employing machine learning models on the Apache PySpark platform. The system uses large scale air quality datasets for prediction purposes through data preprocessing, feature extraction, and classification algorithm application.

Two datasets are used as input datasets to the proposed system:

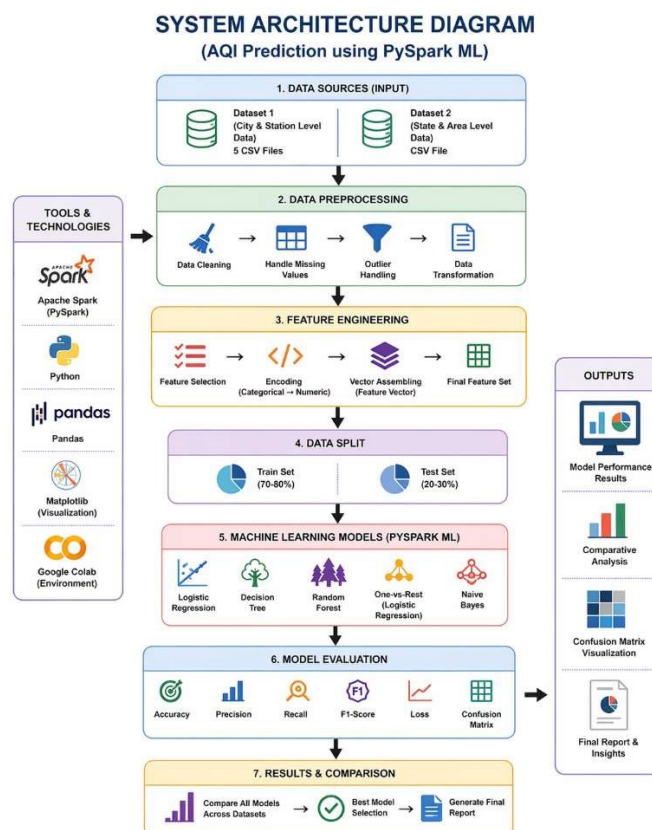
- Dataset 1: Pollutants' detailed dataset with both city level and station level details with granularity of hours and days.
- Dataset 2: Aggregate AQI Dataset with details such as state level, areas, and major pollutants.

The whole process followed by the proposed system consists of:

- Data ingestion through CSV files
- Data Cleaning and preprocessing
- Feature engineering
- Model training through various classification algorithms
- Model Evaluation and Comparison

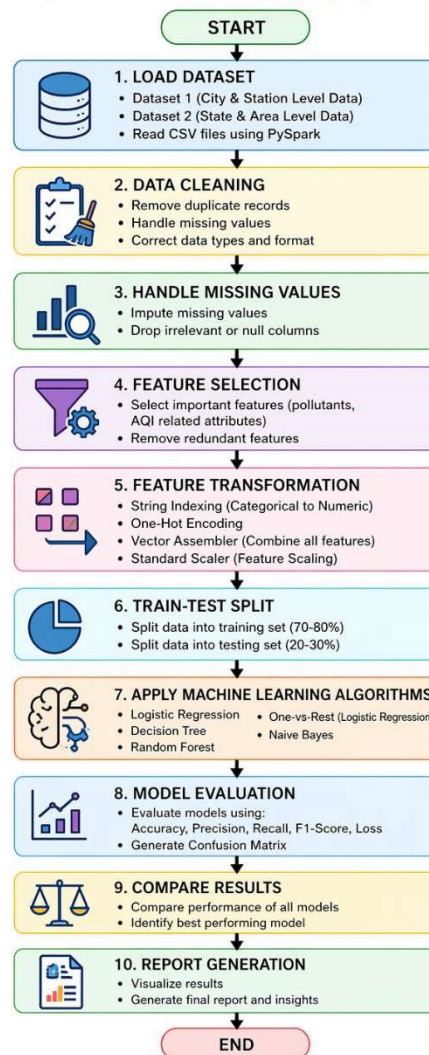
The system leverages PySpark's distributed processing capability to efficiently handle large datasets and improve computation speed.

2.2 System Architecture Diagram



2.3 Proposed Methodology Flow Chart

PROPOSED METHODOLOGY FLOW CHART
(AQI Prediction using PySpark ML)



2.4 Algorithms Used

The project uses five machine learning classification algorithms implemented using PySpark MLlib:



1. Logistic Regression

A model applied in classification where the algorithm computes probabilities and provides exact predictions provided the data is linearly separable.

2. Decision Trees

It is a tree-like algorithm where data is sorted out in branches according to their characteristics. It is easy to interpret but can suffer from overfitting.

3. Random Forest

It is an ensemble machine learning algorithm where several decision trees are combined together.

4. One-vs-Rest (Logistic Regression)

Applied in multi-class classification tasks where each class of data is considered individually in a two-class prediction problem.

5. Naïve Bayes

Classification algorithm based on Bayesian theorem where independence between predictors is assumed.

2.5 Tools and Technologies Used

The success of the project depends on the use of a number of programming languages, frameworks, and big data processing tools.

Programming Language Python: Python is chosen because it is an easy-to-use and well-readable programming language with plenty of libraries for data science and machine learning.

Frameworks Apache Spark (PySpark + MLlib): Apache Spark is a distributed computing framework for efficient large scale data processing. In our work, we use PySpark – Python library for Apache Spark. Spark MLlib provides scalable classification algorithms for our machine learning models.

Development Platform Google Colab: We use Google Colab as our platform for developing and testing the software application as it offers a cloud computing environment with free computational resources.

Data Storage Format CSV: All the data sets are stored in CSV format.



2.6 Experimental Setup

The experimental setup for this study includes the processing of large-scale AQI datasets and creating machine learning models using a distributed computing paradigm. Such tasks will be realized within the scope of Google Colab, which offers cloud environments for performing dataintensive operations and creating machine learning models.

The main programming languages to be used during the realization of this project include Python and PySpark, which stands for the Python API for Apache Spark. It should be noted that PySpark is a tool that helps in carrying out fast data analysis thanks to its distributed computing abilities. Thus, data pre-processing (cleaning, handling missing values, transforming the data, etc.) will be conducted via Spark DataFrames.

In addition, the system will use the distributed computing abilities of Spark to process several types of datasets, including pollutant-level data and AQI data. Encoding techniques and feature vector assembly can be used for feature engineering purposes.

Such classification algorithms as Logistic Regression, Decision Tree, Random Forest, One-vs-Rest and Naive Bayes will be developed using Spark MLlib.

2.7 Dataset Description

This project uses two different datasets to perform Air Quality Index (AQI) classification. These datasets differ in structure, granularity, and feature richness, allowing comparative analysis of machine learning models.

Dataset 1: Detailed Air Quality Dataset

Dataset 1 consists of five CSV files that provide detailed air quality data at both city and station levels:

- city_day.csv – Daily air quality data at the city level
- city_hour.csv – Hourly air quality data at the city level
- station_day.csv – Daily data at the monitoring station level
- station_hour.csv – Hourly data at the station level
- stations.csv – Mapping between cities and monitoring stations



Features:

- Pollutants: PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3
- AQI value and AQI category (AQI_Bucket)

This dataset is rich in features and is suitable for building accurate machine learning models.

Dataset 2: Aggregated AQI Dataset

Dataset 2 is a simplified and aggregated dataset that provides AQI-related information at a broader level.

Features:

- Date
- State
- Area (City/Region)
- Number of monitoring stations
- Prominent pollutant
- AQI value
- Air quality category (Good, Satisfactory, Moderate, etc.)

This dataset is less complex and is useful for comparing model performance on simpler data.

Chapter 3

PROJECT IMPLEMENTATION

3.1 Development Environment

Development and deployment of the proposed system will be carried out on Google Colab, a platform designed for running Python code through the browser. This specific platform was chosen since it saves users' time on setting up the environment locally and provides access to free computing power (CPU/RAM) that can be used to handle moderately-large-sized datasets.



The system will be developed using Python 3.x version along with PySpark, the most relevant library that supports operations related to distributed analysis and machine learning. Configuration of the Apache Spark platform will allow utilizing its capabilities to distribute computations and effectively process big datasets considering issues related to air pollution.

Processing of the information will be performed by applying Spark DataFrames, which possess some benefits such as filtering, transformations, and aggregation. The Pandas library will be used to analyze the dataset at first and plot data using the Matplotlib library.

These components create a favorable environment for developing machine learning solutions.

3.2 Module Description

The project is divided into several functional modules, each responsible for a specific task in the AQI prediction pipeline.

The system is modularized into various functional blocks that perform different tasks in the prediction process.

1. Data Loading Module

The first module is used to import data files in PySpark. The CSV files of both datasets 1 and 2 are imported in PySpark environment as spark dataframes.

2. Data Preprocessing Module

This module consists of cleaning and pre-processing of the data. The missing values are appropriately imputed or removed depending on their occurrence frequency. Duplicates are removed, and incorrect data types are corrected.

3. Feature Engineering Module

This module is responsible for converting raw data into a useful and usable form. This involves converting categorical data like cities and pollutants' names to numbers via string indexing technique. The feature vectors are formed by VectorAssembler which joins multiple columns of inputs.

4. Model Training Module



In this module, we develop various machine learning algorithms using the PySpark MLlib package. These models include Logistic regression, decision tree, random forest, One-vs-Rest classifier, and naive bayes.

5. Model Evaluation Module

The model evaluation module is responsible for evaluating the performance of individual models based on criteria like accuracy, precision, recall, and F1-score. The analysis of confusion matrices is also performed to evaluate the performance of the classifier.

6. Results Analysis Module

This module performs comparison between the performance of different models over the two datasets. Visualization techniques are employed to represent the results.

3.3 Code Implementation

The deployment of the project is done through PySpark in Google Colab in a systematic fashion.

First, the necessary packages and Spark session are imported. The dataset files are uploaded and then converted to Spark DataFrames. After this, preprocessing of the data is done to ensure that any missing values or inconsistencies are taken care of.

The process of feature engineering takes place next by transforming all the categorical variables into numeric ones. Then, the vector assembler classifies the dataset into feature vectors.

The dataset is split into training and testing parts with the ratio of 70:30. Different types of ML algorithms are applied to the data using PySpark MLlib package. The training and prediction take place with the application of various algorithms.

Finally, the predictions are evaluated and analyzed, and results from different algorithms are saved to check which one produces the best results.

3.4 System Interface / Screenshots

The user interface comprises the Google Colab notebook, wherein all the tasks associated with the project are performed. The interface includes the use of code cells as well as the output generated.

The below screenshots are part of this report:

- Output of Dataset Loading: The structure and preview of the datasets are shown here.

```
df1 = spark.read.csv("aqi.csv", header=True, inferSchema=True)
df1.show(5)
```

date	state	area	number_of_monitoring_stations	prominent_pollutants	aqi_value	air_quality_status
30-04-2025	Maharashtra	Amravati	2	PM10	78	Satisfactory
30-04-2025	Bihar	Purnia	1	CO	56	Satisfactory
30-04-2025	Madhya Pradesh	Katni	1	O3	98	Satisfactory
30-04-2025	Chhattisgarh	Tumidih	1	PM10	103	Moderate
30-04-2025	Assam	Byrnihat	1	PM2.5	61	Satisfactory

only showing top 5 rows

```
df2 = spark.read.csv("city_day.csv", header=True, inferSchema=True)
df2.show(5)
```

City	Datetime	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket
Delhi	2015-01-01	153.3	241.7	182.9	33.0	81.3	38.5	1.87	64.5	83.6	18.93	20.81	8.32	204.5	Severe
Mumbai	2015-01-01	70.5	312.7	195.0	42.0	122.5	31.5	7.22	83.8	108.0	2.01	19.41	2.86	60.9	Satisfactory
Chennai	2015-01-01	174.1	275.4	56.2	68.8	230.9	28.5	8.56	60.8	43.9	19.07	10.19	9.63	486.5	Severe
Kolkata	2015-01-01	477.2	543.9	14.1	76.4	225.9	45.6	2.41	42.1	171.1	9.31	11.65	9.39	174.4	Very Poor
Bangalore	2015-01-01	171.6	117.7	123.3	12.4	61.9	49.7	1.26	79.7	164.3	6.04	12.74	9.59	489.7	Good

only showing top 5 rows

- Output of Data Preprocessing: The cleansed and modified data is shown here.

```
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml import Pipeline

# Fill missing values (IMPORTANT)
df1 = df1.fillna({
    "state": "Unknown",
    "area": "Unknown",
    "prominent_pollutants": "Unknown",
    "air_quality_status": "Unknown",
    "number_of_monitoring_stations": 0
})

cat_cols = ["state"]

indexers = [StringIndexer(inputCol=c, outputCol=c+"_idx", handleInvalid="keep") for c in cat_cols]

label_indexer = StringIndexer(inputCol="air_quality_status", outputCol="label")

feature_cols = ["number_of_monitoring_stations"] + [c+"_idx" for c in cat_cols]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

pipeline = Pipeline(stages=indexers + [label_indexer, assembler])

df1 = pipeline.fit(df1).transform(df1)
```

- Output of Feature Engineering: This screenshot shows the indexing and assembly of the features.

```
cat_cols = ["state"]

indexers = [StringIndexer(inputCol=c, outputCol=c+"_idx", handleInvalid="keep") for c in cat_cols]

label_indexer = StringIndexer(inputCol="air_quality_status", outputCol="label")

feature_cols = ["number_of_monitoring_stations"] + [c+"_idx" for c in cat_cols]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

pipeline = Pipeline(stages=indexers + [label_indexer, assembler])

df1 = pipeline.fit(df1).transform(df1)
```

- Output of Model Training: The result of several machine learning models

```
results = []

for name, model in models.items():

    # Dataset 1
    m1 = model.fit(train1)
    pred1 = m1.transform(test1)
    acc1, p1, r1, f11, loss1 = evaluate(pred1)

    # Dataset 2
    m2 = model.fit(train2)
    pred2 = m2.transform(test2)
    acc2, p2, r2, f12, loss2 = evaluate(pred2)

    results.append((name, "Dataset1", acc1, p1, r1, f11, loss1))
    results.append((name, "Dataset2", acc2, p2, r2, f12, loss2))

results

0.15633777942634913,
0.8341219037016421),
('Decision Tree',
'Dataset1',
0.42920700832609476,
0.40117586168944297,
0.4292070083260948,
0.36911793895398876,
0.5707929916739052),
('Decision Tree',
'Dataset2',
0.168939604787086,
0.15505444667492171,
0.168939604787086,
0.1398339958016767,
0.831060395212914),
```

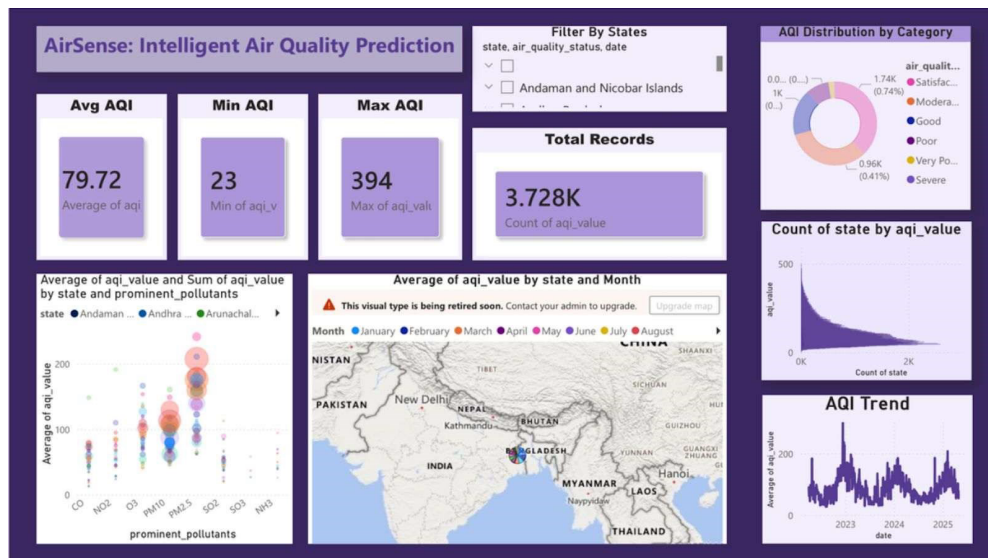
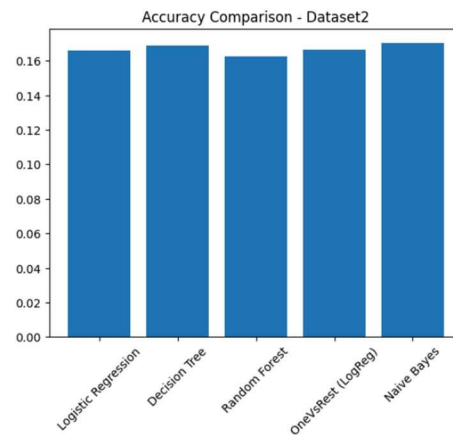
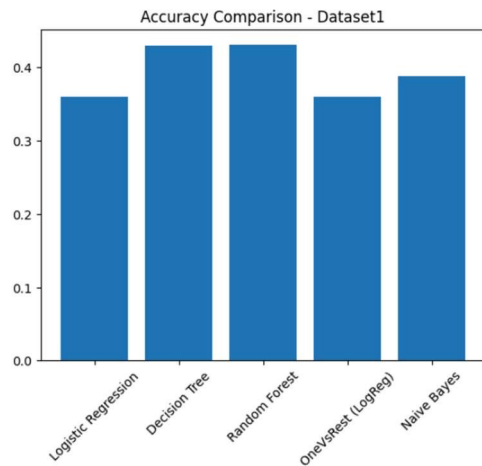
- Evaluation Metrics: The accuracy, precision, recall, and F1 scores for each model

	Model	Dataset	Accuracy	Precision	Recall	F1	Loss	
0	Logistic Regression	Dataset1	0.359039	0.243709	0.359039	0.275068	0.640961	
1	Logistic Regression	Dataset2	0.165878	0.167553	0.165878	0.156338	0.834122	
2	Decision Tree	Dataset1	0.429207	0.401176	0.429207	0.369118	0.570793	
3	Decision Tree	Dataset2	0.168940	0.155054	0.168940	0.139834	0.831060	
4	Random Forest	Dataset1	0.429906	0.394790	0.429906	0.378906	0.570094	
5	Random Forest	Dataset2	0.162538	0.164970	0.162538	0.155292	0.837462	
6	OneVsRest (LogReg)	Dataset1	0.359039	0.243709	0.359039	0.275068	0.640961	
7	OneVsRest (LogReg)	Dataset2	0.166435	0.168631	0.166435	0.156732	0.833565	
8	Naive Bayes	Dataset1	0.387640	0.340833	0.387640	0.328335	0.612360	
9	Naive Bayes	Dataset2	0.170053	0.172123	0.170053	0.164273	0.829947	

- Graphical Visualizations: Graphical representation using bar charts showing the comparison among the models.



THE TIMES GROUP





Chapter 4

PROJECT RESULTS AND DISCUSSION

4.1 Experimental Setup

The experiment set-up for the current research focuses on evaluating different machine learning models' performance on the air quality data sets through distributed computing. The experiment is implemented in a Google Colab environment with the use of Python and PySpark. In this regard, the data sets are processed by applying Spark DataFrames for working with large datasets effectively.

To prepare the data sets, two approaches have been utilized. Thus, Dataset 1 (pollutants data) and Dataset 2 (AQI data) are preprocessed to eliminate missing values, inconsistencies, and formatting errors. Feature engineering methods are employed to transform categorical features into numeric ones and compile a feature vector for training machine learning models.

The data sets are split into the training and testing datasets with a proportion of 70:30 to make an unbiased evaluation of the machine learning models. Specifically, five classification models are selected for this purpose. These are Logistic Regression, Decision Tree, Random Forest, One-vs-Rest, and Naive Bayes.

4.2 Output Results

From the obtained results from our projects, it is clear how effective machine learning algorithms are at classifying AQI levels. In relation to Dataset 1, where all the data is broken down at the pollutant level, most of the algorithms performed quite well. Out of all the algorithms used, Random Forest was more effective, followed by Decision Tree and Logistic Regression.

In contrast to Dataset 1, which included more features, Dataset 2 contained less data, leading to relatively poor results from the models. Even in Dataset 2, Random Forest and One-vs-Rest performed much better compared to the others.

One of the algorithms that failed to give optimal results for both the datasets is Naive Bayes, whose algorithm assumes independent features, making it ineffective. Similarly, another algorithm that did not work out very well was Logistic Regression, which could not capture the non-linearities of the datasets.

It is therefore clear that detailed models perform better than aggregated models.

4.3 Performance Analysis

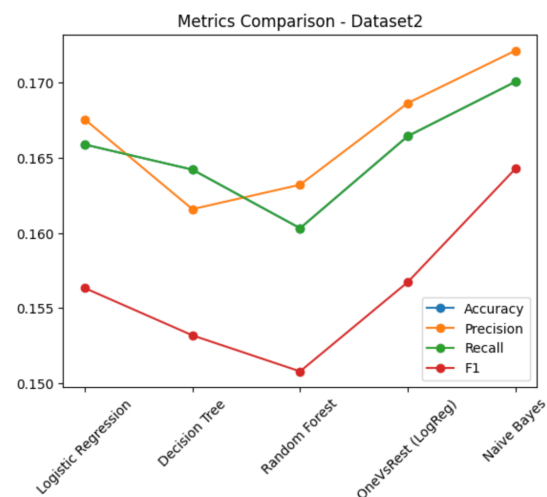
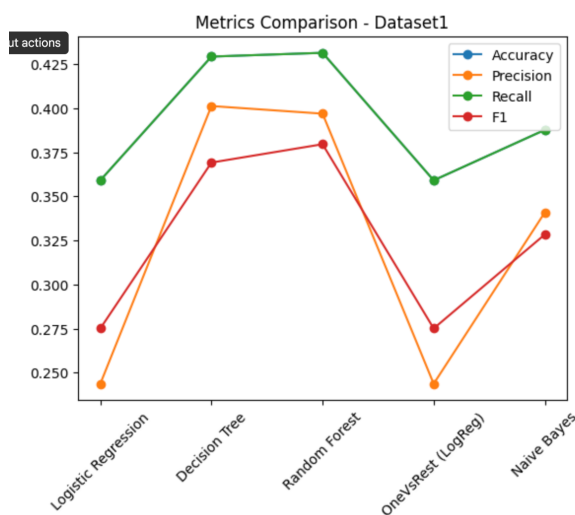
The performance of the models was assessed using several different performance measures, namely accuracy, precision, recall, and F1-score. Random Forest showed the best result in both data sets, suggesting its flexibility and adaptation to the complex relations.

Decision Tree showed also quite a good result, although there was some problem with overfitting on the part of this model when applied to the first data set. Logistic Regression offered steady but moderate performance because this algorithm makes an assumption about linear correlations between the features.

One-vs-Rest made Logistic Regression perform better in multi-class classification problems, showing slightly better results compared to Logistic Regression in relation to the second data set, while still remaining worse than Random Forest.

Naive Bayes proved to be the worst among other models because of their assumptions that simplify the situation. The analysis of the result showed that the number of features and data complexity had a significant impact on performance.

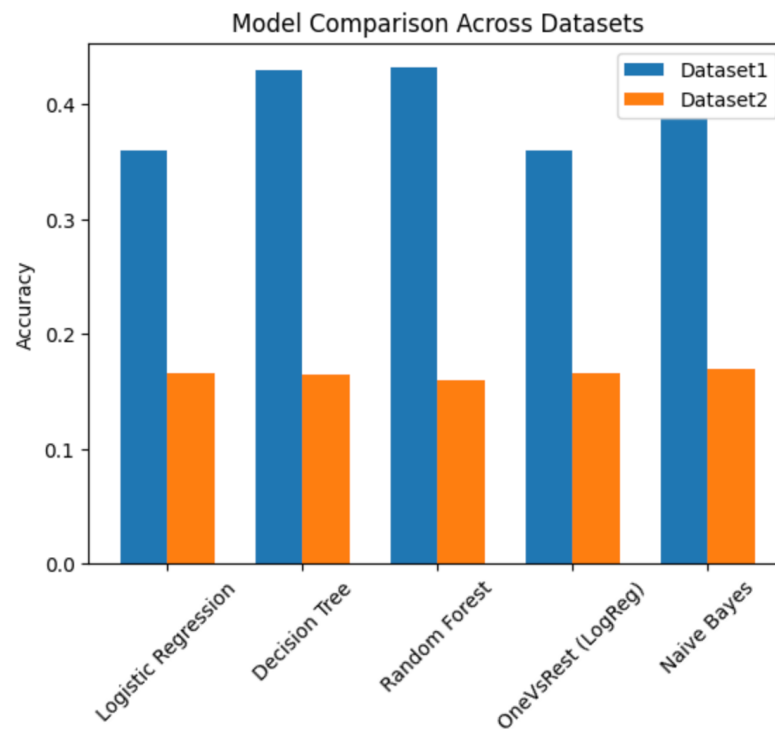
Visual depiction of the results was provided by bar graphs.



4.4 Discussion

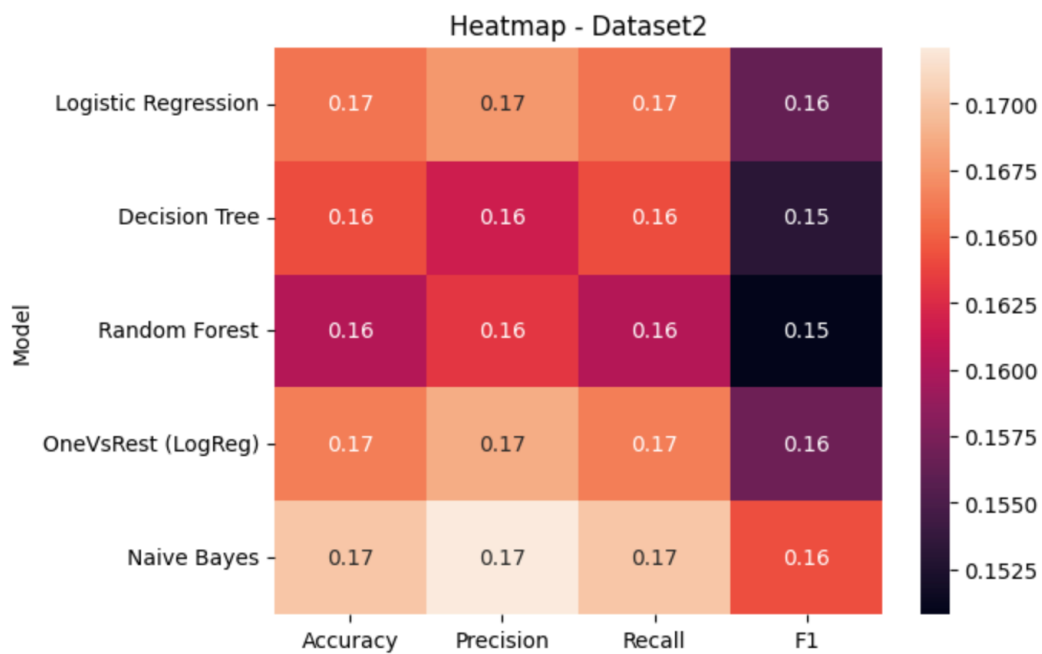
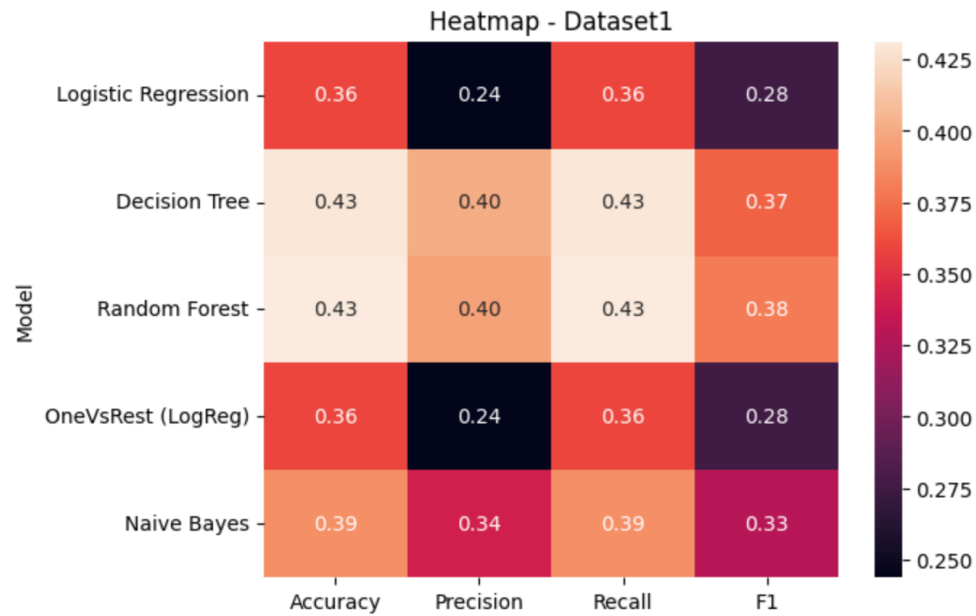
From the results of this experiment, it can be seen that high-quality data sets and richer features can help develop an effective AQI prediction system using machine learning algorithms. This is because there is rich data available in Dataset 1; thus, any machine learning algorithm created based on this dataset will make better predictions. In contrast, Dataset 2 is more generalized; therefore, any machine learning algorithm based on this data set will fail in predicting future values.

Richer data makes machine learning algorithms perform better. From the results, it can be observed that the use of ensemble methods like Random Forest in the process of creating machine learning algorithms improves the accuracy of predictions because they can capture relationships among features. The effectiveness of PySpark in big data analytics has been demonstrated.





THE TIMES GROUP



Chapter 5

PROJECT CONCLUSION AND PRESENTATION

5.1 Conclusion

This study has successfully shown how machine learning can be used to classify the Air Quality Index based on extensive air quality data sets. This was made possible by utilizing the capabilities offered by the Apache PySpark framework. Through the implementation and testing of various machine learning algorithms, such as Logistic Regression, Decision Tree, Random Forest, One-vs-Rest, and Naive Bayes, this project has identified the most efficient algorithm. Random Forest, due to its flexibility in handling complicated data patterns, proved to be the optimal solution. It was also found that data sets consisting of detailed data about pollutants lead to better predictions than aggregating data sets.

In summary, it can be said that the project demonstrated the significance of scalable data processing tools in solving problems related to air quality. Using the Apache PySpark library for this purpose was proven to be highly effective.

5.2 Limitations

However, there are also some drawbacks in spite of the positive results. Datasets used in this research have missing values and discrepancies that might impact the model accuracy. Though, data pre-processing was performed, there could be some data quality problems left.

The project covers only the classification task for AQI groups and doesn't imply regression forecasting of precise AQI values. In addition, deep learning algorithms and time series forecasting were not considered.

Only offline processing of the data set was realized, but no online AQI prediction was implemented. Moreover, hyper-parameter tuning was not carried out.

5.3 Future Scope

There are numerous possibilities for developing and enhancing this project in the future. One of the possible ways of improvement could be the inclusion of real-time data collected by IoT devices and APIs in order to predict AQIs in real-time.



One may apply machine learning and deep learning methods, for example, the LSTM algorithm, to perform time-series forecasting. In addition, one can add other types of data, such as meteorological data, which includes temperature, humidity, and wind speed.

Furthermore, hyperparameter tuning and model optimization techniques can be utilized in order to optimize the model. Finally, the developed model can be presented as a web or mobile application, making it easily accessible to users.

In addition, one can expand the training set by including additional cities and regions, which would make the developed model more scalable. Besides, one may integrate the system with some visualizations tool like Power BI.

5.4 References

1. Central Pollution Control Board (CPCB), India – Air Quality Index Guidelines
2. Apache Spark Documentation – <https://spark.apache.org>
3. PySpark MLlib Documentation – Machine Learning Library for Spark
4. Research papers on AQI prediction using Machine Learning
5. Google Colab Documentation – <https://colab.research.google.com>